

INTERNSHIP REPORT

submitted in partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications

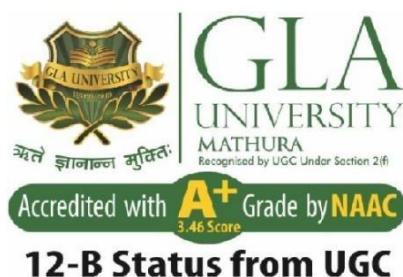
By

Ritik Gaur

2484200162

Under the Guidance of
Mr. Praveen Eruvanti

Department of Computer Engineering & Applications
Institute of Engineering & Technology



GLA University
Mathura- 281406, INDIA
May, 2026

Declaration

I hereby declare that the work which is being presented in the MCA Project "Test Automation for Travel Agency Web Application" is an authentic record of my own work carried out during the period of internship at EPAM Systems under the guidance of Mr. Praveen Eruvanti (Industry Mentor/SME).

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign _____

Name of Student: Ritik Gaur

University Roll No.: 2484200162

Certificate

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to EPAM Systems for providing me with the opportunity to undertake this internship. This experience has been instrumental in developing my skills in software test automation and quality assurance engineering.

I am deeply thankful to my SME, Mr. Praveen Eruvanti, for his unwavering mentorship. His clear guidance on test strategy, automation frameworks, and industry best practices helped me grow as a professional. I also extend my gratitude to Mr. Harishankar Chandrasekharan, our Scrum Master, for his leadership in managing the Agile process.

Furthermore, I extend my thanks to the Department of Computer Engineering and Applications, Institute of Engineering and Technology, GLA University, Mathura, and my university guide Mr. Masood Ahmed for their continuous support and encouragement throughout the internship period.

Sign

Name of Candidate: Ritik Gaur

University Roll No: 2484200162

ABSTRACT

This report provides a comprehensive account of a Software Test Automation Engineer internship completed at EPAM Systems. The internship followed a structured, competency-based training model consisting of six learning modules — Java Programming, Design Patterns and Principles, TestNG, REST Assured, Selenium, and BDD Cucumber — each culminating in a capstone project. Following the learning phase, a team-based project was undertaken involving the development of a test automation framework for a Travel Agency Web Application.

The Travel Agency project involved a team of 10 engineers (7 developers and 3 automation testers) following industry-grade Agile methodology with bi-weekly sprints. The application was built using Spring Boot, React, and MongoDB, deployed via KubeRocketCI and ArgoCD, with plans for migration to AWS Serverless architecture. The QA responsibilities included creating Test Strategy and Test Plan documentation, designing and implementing automated API tests using REST Assured, and building UI automation tests using Selenium WebDriver.

Beyond functional test automation, the internship emphasized enterprise-grade engineering standards including proper test framework architecture (Maven + Java + TestNG), CI/CD integration, BDD practices with Cucumber, and adherence to Agile processes. This report documents the learning modules, the project work, tools and technologies used, key skills acquired, challenges faced, and areas identified for further exploration.

CONTENTS

Declaration	ii
Certificate	ii
Acknowledgement	iii
Abstract	iv
List of Figures	v
List of Tables	vi
CHAPTER 1 Introduction	1
1.1 About the Organization – EPAM Systems	9
1.2 Nature and Structure of the Internship	10
1.3 Role and Responsibilities	11
CHAPTER 2 Objectives of the Internship	12
2.1 Learning Objectives	12
2.2 How Objectives Were Achieved	12
CHAPTER 3 Assignments and Work Performed	14
3.1 Module 1 – Java Programming Language (5 assignments)	15
3.2 Module 2 – TestNG – testing framework (2 assignments)	17
3.3 Module 3 – Rest Assured – API testing framework (Capstone project)	20
3.4 Module 4 – Selenium – UI Testing Framework (Capstone project)	22
3.5 Module 5 – BDD Cucumber for behavior driven development	25
CHAPTER 4 Tools and Technology Stack	31
CHAPTER 5 Key Skills Learned	34
CHAPTER 6 Challenges Faced	36
CHAPTER 7 Topics to Explore Further	38
CHAPTER 8 Conclusion	40
APPENDICES	
Appendix A Assignment Submission Details	42
Appendix B Cloud Infrastructure Used	42
Appendix C Security Practices Followed	42
Appendix D Abbreviations	43
References	44

List of Figures

3.2 Client-Server API Architecture – Bronze, Silver, Gold Layers	17
3.3 REST Assured Test Framework Architecture	18
3.4 Selenium WebDriver Architecture	20
3.6 Travel Agency Application Architecture	22
3.7 Agile Scrum Framework Flow	25
3.8 Test Automation Pyramid	27
4.1 Tools and Technologies Used During Internship	31
6.1 Internship Experience Journey	37
8.1 Internship Learnings Summary	41

List of Tables

1.1 EPAM Systems – Organization Details	9
1.2 Internship Details	10
3.1 Learning Module Summary Table	14
3.2 Module 1 (Java) – Technologies Used	16
3.3 Module 2 (Design Patterns) – Technologies Used	18
3.4 Module 3 (TestNG) – Technologies Used	21
3.5 Module 4 (REST Assured) – Technologies Used	23
4. Chapter 4	31

Chapter 1

Introduction

1.1 About the Organization – EPAM Systems

EPAM Systems is a leading global digital engineering and IT services company founded in 1993 and headquartered in Newtown, Pennsylvania, USA. Listed on the New York Stock Exchange (NYSE) since 2012, EPAM is recognized worldwide for its engineering-first culture, delivery quality, and ability to build scalable, high-performance technology platforms for enterprise clients across a wide range of industries.

The organization provides end-to-end technology services spanning software engineering, cloud computing, data engineering, artificial intelligence, DevOps, UI/UX design, cybersecurity, and strategic consulting. EPAM serves clients in sectors including banking and financial services, healthcare, retail, telecommunications, manufacturing, media, and travel.

Founded	1993
Headquarters	Newtown, Pennsylvania, USA
Listed On	New York Stock Exchange (NYSE) – 2012
Core Services	Software Engineering, Cloud, Data, AI, DevOps, Cybersecurity, Consulting
Global Presence	North America, Europe, Asia-Pacific, Latin America

EPAM promotes a collaborative, mentor-driven work culture where learning is structured and continuous. The organization follows agile delivery methodologies, encourages knowledge sharing, and provides employees and interns with access to industry-standard tools, structured training programs, and experienced technical mentors.

1.2 Nature and Structure of the Internship

The internship at EPAM Systems followed a structured, module-based training model. The program consisted of two phases: (1) Six sequential learning modules covering Java, Design Patterns, TestNG, REST Assured, Selenium, and BDD Cucumber — each with a capstone project; and (2) A team-based Agile project developing test automation for a Travel Agency Web Application.

Each learning module came with structured content, hands-on exercises, and a capstone project to validate competency. The team project followed industry-standard Agile methodology with bi-weekly sprints, daily standups, sprint planning, and retrospectives under the guidance of Scrum Master Harishankar Chandrasekharan.

This model closely mirrors how structured QA engineering training operates in leading technology organizations — building skills incrementally through focused modules before applying them in a real team project environment.

Student Name	Ritik Gaur
Enrollment Number	2484200162
Program	MCA
Organization	EPAM Systems
Role	Software Test Automation Engineer Intern
Guide	Mr. Praveen Eruvanti (SME)
Department	Department of Computer Engineering & Applications
Duration	Approximately 10–12 weeks (6 learning modules + team project) 10–12 weeks
Mode	Structured learning modules with capstone projects + Agile team project assignments with real tools and Azure cloud infrastructure

1.3 Role and Responsibilities

The designated internship role was Software Test Automation Engineer Intern. The responsibilities associated with this role included:

- Completing structured learning modules on Java, Design Patterns, TestNG, REST Assured, Selenium, and BDD Cucumber
- Building capstone projects for each module demonstrating practical competency
- Creating Test Strategy and Test Plan documentation for the Travel Agency project
- Designing and implementing automated API tests using REST Assured with Java
- Building UI automation test suites using Selenium WebDriver with Page Object Model
- Participating in Agile ceremonies — sprint planning, daily standups, sprint reviews, and retrospectives
- Reviewing QA workshops and applying best practices in test automation
- Collaborating with developers and other automation testers in a team of 10 engineers

Chapter 2

Objectives of the Internship

2.1 Learning Objectives

The internship was designed with the following learning objectives, each aligned to one or more specific modules or project phases:

1. Build foundational and advanced skills in Java programming for test automation
2. Understand and implement software design patterns and SOLID principles for maintainable test code
3. Gain hands-on experience with TestNG framework for structured test execution and reporting
4. Implement API test automation using REST Assured for validating RESTful web services
5. Learn Selenium WebDriver for browser-based UI test automation with Page Object Model
6. Apply BDD methodology using Cucumber for writing human-readable test scenarios
7. Develop test strategy and test planning skills for enterprise-grade applications
8. Work in an Agile team environment following Scrum framework with bi-weekly sprints

2.2 How Objectives Were Achieved

Each objective was addressed through the structured learning modules and the team-based project. The program ensured practical mastery through:

- Completing capstone projects for each of the 6 learning modules with passing scores
- Building a complete test automation framework using Maven, Java, TestNG, REST Assured, and Selenium
- Creating comprehensive Test Strategy and Test Plan documents following industry standards
- Writing automated API test cases for the Travel Agency backend REST endpoints
- Implementing UI automation tests for critical user flows using Selenium WebDriver
- Following Agile practices with sprint-based delivery and regular demo sessions

Chapter 3

Assignments and Work Performed

The internship comprised two phases: six structured learning modules (each with a capstone project) and a team-based Agile project. The learning modules built foundational and advanced test automation skills incrementally. The team project applied these skills to a real-world Travel Agency Web Application.

#	Module Title Title	Core Technology
1	Java Programming Language	Core Java, OOP, Collections, Streams
2	Design Patterns and Principles	SOLID, Creational, Structural, Behavioral Patterns
3	TestNG Framework	Test Annotations, Assertions, Data Providers, Reporting
4	REST Assured API Testing	HTTP Methods, Request/Response Validation, API Automation
5	Selenium WebDriver	Web Automation, Page Object Model, WebDriver API
6	BDD Cucumber	Gherkin Syntax, Step Definitions, Feature Files

3.1 Module 1 – Java Programming Language

3.1.1 Objective

The objective was to build strong foundational skills in Java programming language covering core concepts, object-oriented programming, collections framework, exception handling, streams API, and writing clean, maintainable code for test automation.

3.1.2 Tasks Completed

1. Learned core Java fundamentals — variables, data types, operators, control flow, and loops
2. Implemented object-oriented programming concepts — classes, objects, inheritance, polymorphism, abstraction, and encapsulation
3. Worked with Java Collections Framework — List, Set, Map, Queue and their implementations
4. Practiced exception handling with try-catch, custom exceptions, and proper error management
5. Used Java Streams API for functional-style data processing and transformations
6. Wrote utility classes and helper methods following clean code principles
7. Built a capstone project demonstrating practical application of all Java concepts learned

3.1.3 Technologies Used

Java 17	Language	Core programming language for all automation
IntelliJ IDEA	IDE	Development environment
Maven	Build Tool	Project build and dependency management
JUnit/TestNG	Testing	Unit testing during capstone
Git	Version Control	Code versioning and submission

3.1.4 Key Learnings

- How object-oriented design principles apply to building test automation frameworks
- How geohash-based spatial joins work and why 4-character precision is suitable for matching
- How to write modular, reusable Java code suitable for automation projects
- How to package a Spark application as a Docker image and deploy it on Kubernetes via spark-submit

3.2 Module 2 – Design Patterns and Principles

3.2.1 Objective

The objective was to understand and implement software design patterns and SOLID principles that are essential for building maintainable, scalable test automation frameworks.

Figure 3.2: Client-Server API Architecture – Bronze, Silver, Gold Layers

3.2.2 Tasks Completed

1. Studied SOLID principles — Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion
2. Implemented Creational patterns — Singleton, Factory, Builder, and Abstract Factory for organizing test suites and configurations
3. Applied Structural patterns — Adapter, Decorator, Facade, and Proxy
4. Practiced Behavioral patterns — Strategy, Observer, Command, and Template Method
5. Learned how design patterns apply to test automation framework architecture (e.g., Page Object Model, Factory Pattern for drivers)

Figure 3.3: REST Assured Test Framework Architecture

3.2.3 Technologies Used

Java 17	Language	Implementation of design patterns
UML Diagrams	Design	Visualizing pattern structures
Maven	Build Tool	Project management
IntelliJ IDEA	IDE	Development environment

TestNG	Test Framework	Test execution and configuration
Maven Surefire Plugin	Build Plugin	TestNG integration with Maven
testng.xml	Configuration	Suite and test configuration

3.2.4 Key Learnings

- How SOLID principles lead to maintainable and extensible test automation code
- How the Factory pattern simplifies WebDriver instantiation for cross-browser testing
- How the Strategy pattern enables flexible test execution strategies
- How the Page Object Model pattern (a variation of Facade) improves UI test maintainability

3.3 Module 3 – TestNG Framework

3.3.1 Objective

The objective was to master the TestNG testing framework for Java, covering test annotations, assertions, test configuration, data-driven testing, parallel execution, test grouping, and reporting capabilities.

Figure 3.4: Selenium WebDriver Architecture

3.3.2 Tasks Completed

1. Learned TestNG annotations — @Test, @BeforeMethod, @AfterMethod, @BeforeClass, @AfterClass, @BeforeSuite, @AfterSuite
2. Implemented assertions for test validation — assertEquals, assertTrue, assertNotNull, and soft assertions
3. Used DataProvider for data-driven testing with multiple test data sets
4. Configured test suites using testng.xml for organized test execution
5. Implemented parallel test execution for improved test run efficiency
6. Used test groups and dependencies for logical test organization
7. Generated and customized TestNG reports for test execution visibility

Built a capstone project with a complete TestNG test suite demonstrating all learned concept

3.3.3 Technologies Used

REST Assured	API Testing	HTTP request/response automation
Hamcrest	Matchers	Response body assertion
Jackson/Gson	Serialization	JSON to POJO mapping
Postman	Manual Testing	API exploration and validation
TestNG	Test Framework	Test execution and reporting
Maven	Build Tool	Dependency and build management

3.3.4 Key Learnings

- How TestNG provides superior test configuration and execution control compared to JUnit
- How data-driven testing with DataProviders enables comprehensive test coverage
- How parallel execution significantly reduces test suite execution time
- How proper test suite organization with groups and dependencies improves maintainability

3.4 Module 4 – REST Assured API Testing

3.4.1 Objective

The objective was to master API test automation using REST Assured library in Java, covering HTTP methods, request/response specification, authentication, serialization, JSON/XML parsing, and building comprehensive API test suites.

Figure 3.6: Travel Agency Application Architecture

3.4.2 Tasks Completed

1. Learned REST Assured fundamentals — given/when/then BDD-style syntax for API testing
 2. Implemented tests for all HTTP methods — GET, POST, PUT, PATCH, DELETE
 3. Validated response status codes, headers, and body content using Hamcrest matchers
 4. Worked with request and response specifications for reusable test configuration
- Implemented authentication testing — Basic Auth, OAuth2, and API key validation

5. Used JSON Path and XML Path for response body extraction and assertion
6. Applied serialization/deserialization with POJO classes for type-safe request/response handling
7. Built a capstone project with a complete API test suite covering CRUD operations
- 8.

3.4.3 Technologies Used

Selenium WebDriver	UI Automation	Browser interaction and automation
ChromeDriver / GeckoDriver	Browser Drivers	Chrome and Firefox automation
Page Object Model	Design Pattern	Maintainable test architecture
TestNG	Test Framework	Test execution and assertions
WebDriverManager	Driver Management	Automatic driver binary management
Maven	Build Tool	Project and dependency management

3.4.4 Key Learnings

- **Automation Framework Design**
Gained hands-on experience in designing a scalable and maintainable test automation framework (e.g., hybrid, POM-based), ensuring separation of concerns and reusability of components.
- **Selenium WebDriver Proficiency**
Developed deep understanding of Selenium WebDriver APIs, including element locators, synchronization techniques (implicit/explicit waits), and handling dynamic web elements.

- **Test Execution Strategy**
Implemented structured test execution using **TestNG**, including annotations, test grouping, parallel execution, and generation of detailed test reports.
- **BDD Implementation with Cucumber**
Learned Behavior-Driven Development using Cucumber by writing feature files in Gherkin, mapping step definitions, and improving collaboration between technical and non-technical stakeholders.
- **API Automation Integration**
Integrated API testing using **Rest Assured** within the framework, enabling end-to-end validation across UI and backend layers.
- **Dependency Injection & Code Management**
Utilized **PicoContainer** for dependency injection to manage object lifecycle efficiently, reducing code duplication and improving modularity.
- **Error Handling & Reporting**
Implemented robust error handling, logging, and reporting mechanisms (e.g., screenshots on failure, custom logs), improving debugging efficiency and traceability of test failures.

3.5 Module 5 – Selenium WebDriver

3.5.1 Objective

The objective was to master Selenium WebDriver for browser-based UI test automation, covering element locators, WebDriver API, waits, Page Object Model, cross-browser testing, and building robust UI automation frameworks.

3.5.2 Tasks Completed

1. Learned Selenium WebDriver architecture and browser driver management
2. Mastered element locator strategies — ID, Name, CSS Selector, XPath, Link Text
3. Implemented explicit and implicit waits for handling dynamic web elements
4. Built Page Object Model (POM) framework for maintainable UI test scripts
5. Automated form interactions — text input, dropdowns, checkboxes, radio buttons, file uploads
 - Handled advanced interactions — alerts, frames, multiple windows, and mouse/keyboard actions
 - Implemented cross-browser testing with Chrome, Firefox, and Edge WebDrivers
 - Built a capstone project automating end-to-end user flows on a web application

3.5.3 Key Learnings

- How Page Object Model dramatically improves test maintenance and readability
- How explicit waits prevent flaky tests caused by dynamic content loading
- How CSS Selectors and XPath provide flexible element identification strategies
- How cross-browser testing ensures application compatibility across different browsers

3.6 Module 6 – BDD Cucumber

3.6.1 Objective

The objective was to learn Behavior-Driven Development (BDD) using Cucumber framework, covering Gherkin syntax, feature files, step definitions, scenario outlines, hooks, and integrating Cucumber with Selenium and REST Assured for end-to-end test automation.

Figure 3.8: Test Automation Pyramid

3.6.2 Tasks Completed

8. Learned Gherkin syntax — Given/When/Then for writing human-readable test scenarios
 9. Created feature files organizing test scenarios by application functionality
 10. Implemented step definitions in Java linking Gherkin steps to automation code
 11. Used Scenario Outlines with Examples tables for data-driven BDD testing
 12. Configured Cucumber hooks (@Before, @After) for test setup and teardown
- Integrated Cucumber with Selenium WebDriver for UI BDD test automation

13. Integrated Cucumber with REST Assured for API BDD test automation
14. Generated Cucumber HTML reports for clear test execution documentation
15. Built a capstone project with complete BDD test suite covering multiple features

3.6.3 Key Learnings

- How BDD bridges the communication gap between technical and non-technical stakeholders
- How Gherkin syntax enables business-readable test documentation
- How Cucumber integrates with existing automation frameworks (Selenium, REST Assured)
- How Scenario Outlines enable efficient data-driven testing with minimal code duplication

3.7 Team Project – Test Automation for Travel Agency Web Application

3.7.1 Objective

Following the six learning modules, a team of 10 engineers (7 developers and 3 automation testers) was formed to build a Travel Agency Web Application. The application is built using Spring Boot 3 (backend), React (frontend), and MongoDB (database), deployed using KubeRocketCI and ArgoCD on Kubernetes. The frontend is hosted on AWS S3. The backend is planned for migration to AWS Serverless architecture. As one of the 3 automation testers, the role involved creating test strategy, test plan, and implementing automated tests using REST Assured and Selenium.

3.7.2 Tasks Completed

1. Analyzed functional requirements and user stories for Sprint 1 (User Profile and Travel Booking features)
2. Created comprehensive Test Strategy document defining scope, approach, tools, and risk assessment
3. Created detailed Test Plan document with test objectives, schedule, resources, and entry/exit criteria
4. Set up the test automation framework using Maven + Java + TestNG + REST Assured + Selenium
 - Reviewed 4 QA Workshops on test automation best practices and methodologies
 - Created API test cases document covering all REST endpoints from the OpenAPI specification
 - Performed manual testing with Postman to validate API endpoints before automation
 - Implemented automated API tests using REST Assured for User Profile and Tour Booking APIs
 - Participated in bi-weekly sprint demos, daily standups, and sprint retrospectives
5. Collaborated with developers on bug reporting and defect tracking within the Agile workflow

3.7.3 Key Learnings

- How to create industry-standard Test Strategy and Test Plan documents for enterprise applications
- How to compute derived fields from streaming record attributes in real time
- How to apply business categorization logic to streaming data and route enriched output to a new topic
- How CI/CD pipelines (KubeRocketCI, ArgoCD) integrate with test automation for continuous testing

Chapter 4

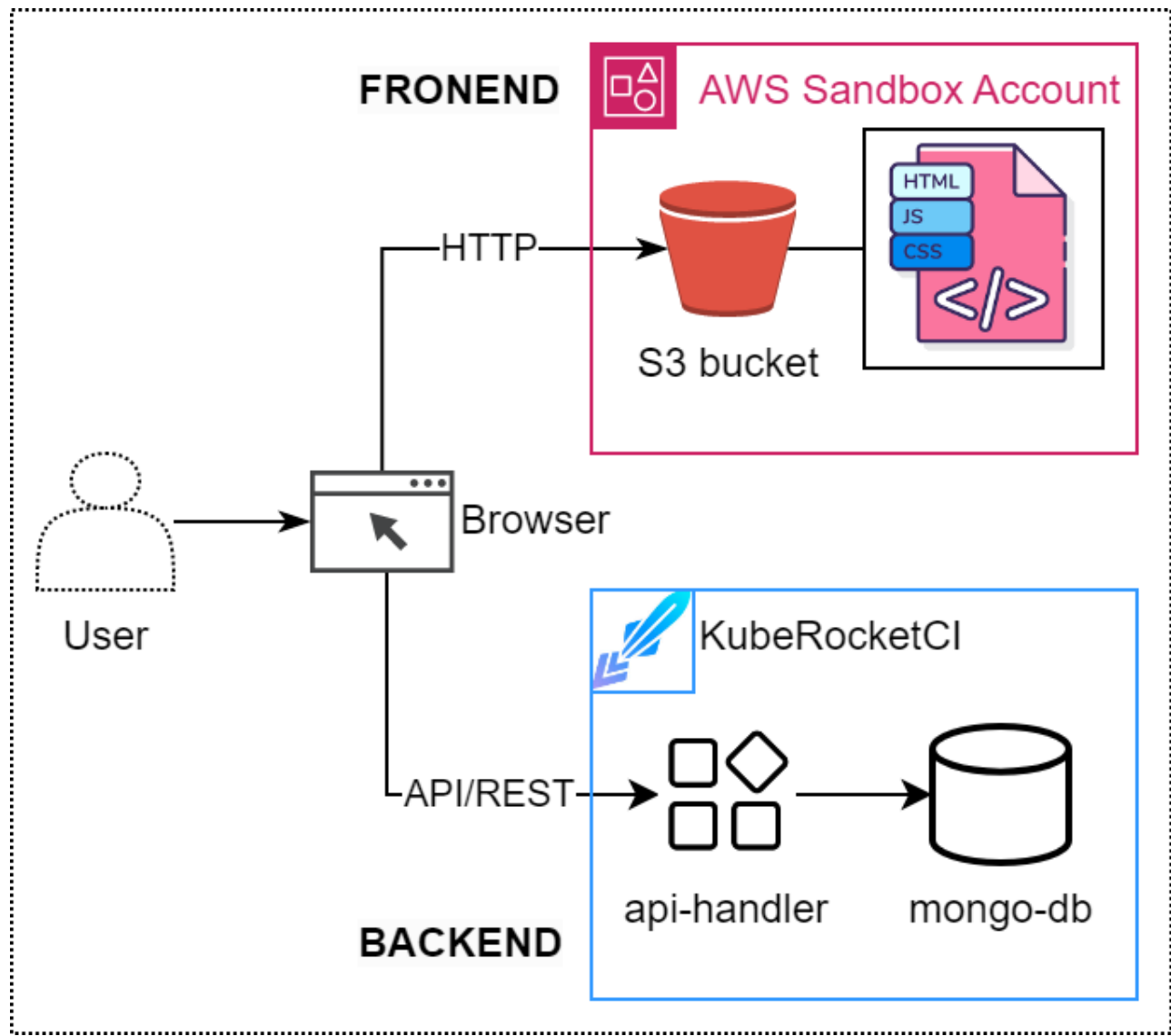
Tools and Technology Stack

The following tools and technologies were actively used, configured, and applied during the six learning modules and the team project of the internship:

Technology	Category	Modules/Project
Java	Programming Language	All Modules + Project
TestNG	Test Framework	Module 3 + Project
REST Assured	API Testing	Module 4 + Project
Selenium WebDriver	UI Testing	Module 5 + Project
Maven	Build Tool	All Modules + Project
Cucumber	BDD Framework	Module 6 + Project
Spring Boot 3	Backend Framework	Project (Application Under Test)
React	Frontend Framework	Project (Application Under Test)
MongoDB	Database	Project (Application Under Test)
KubeRocketCI	CI/CD Platform	Project
ArgoCD	GitOps / CD	Project
AWS S3	Static Hosting	Project (Frontend Deployment)
Kubernetes	Container Orchestration	Project

Postman	API Testing (Manual)	Project
Git / GitHub	Version Control	All
IntelliJ IDEA	IDE	All Modules + Project
Docker	Containerization	Project
Jira	Project Management	Project
Confluence	Documentation	Project
Page Object Model	Design Pattern	Module 5 + Project
Hamcrest Matchers	Assertion Library	Module 4 + Project
ExtentReports	Test Reporting	Project
Gherkin	BDD Language	Module 6
OpenAPI Specification	API Documentation	Project
Agile / Scrum	Methodology	Project
AWS Serverless	Cloud Architecture	Project (Planned Migration)

Deployment Architecture



Chapter 5

Key Skills Learned

5.1 Java Programming and Design Patterns

- Writing clean, modular Java code with proper OOP principles for automation frameworks
- Implementing the Client-Server API Architecture using Delta Lake.
- Using Java Collections and Streams API for efficient test data management
- Building reusable utility classes and helper methods following clean code standards
- Applying exception handling and logging best practices for reliable test execution

5.2 Test Framework and API Testing

- Configuring TestNG for parallel execution, data-driven testing, and comprehensive reporting
- Building REST Assured API test suites with BDD-style given/when/then syntax
- Validating HTTP responses — status codes, headers, JSON body — using Hamcrest matchers
- Implementing request/response specifications for reusable API test configuration

5.3 UI Automation and BDD

- Implementing Selenium WebDriver automation with Page Object Model for maintainable UI tests
- Handling dynamic web elements using explicit waits, JavaScript executor, and advanced interactions
- Writing BDD test scenarios in Gherkin and implementing step definitions in Java

- Integrating Cucumber with Selenium and REST Assured for end-to-end BDD automation
- Generating comprehensive test reports using Cucumber HTML and ExtentReports
- Performing cross-browser testing across Chrome, Firefox, and Edge

5.4 Agile and Project Skills

- Creating Test Strategy and Test Plan documents following industry standards
- Working effectively in Agile Scrum teams with bi-weekly sprints and daily standups
- Collaborating with developers on requirements analysis, bug reporting, and defect tracking
- Participating in sprint planning, reviews, retrospectives, and demo sessions

5.5 CI/CD and DevOps Awareness

- Writing modular reusable code with separate classes for API calls, encryption/decryption, and utility logic
- Familiarity with Docker and Kubernetes for containerized application deployment
- Using Git for version control and collaborative development workflows
- Understanding AWS services (S3, Serverless) for cloud-based application hosting
- Integrating test automation into CI/CD pipelines for continuous testing

Chapter 6

Challenges Faced

6.1 Test Framework Setup Challenges

Setting up the test automation framework from scratch presented several technical challenges:

- Resolving Maven dependency conflicts between different library versions (REST Assured, Selenium, TestNG, Cucumber) requiring careful version management in pom.xml
- Configuring TestNG XML for proper parallel execution without test interference, requiring thread-safe WebDriver management and test isolation
- Setting up proper project structure following Maven conventions and ensuring all dependencies resolve correctly across different development environments

6.2 API Testing Challenges

- Handling dynamic web elements with unpredictable load times — resolved using explicit waits with custom expected conditions instead of Thread.sleep
- Dealing with StaleElementReferenceException in Selenium when DOM updates during test execution — resolved with retry mechanisms and proper wait strategies
- Cross-browser compatibility issues where locators worked in Chrome but failed in Firefox — resolved by using more robust CSS selectors and XPath expressions
- Flaky tests caused by race conditions in asynchronous UI operations — resolved by implementing proper synchronization and fluent waits

6.3 UI Automation Challenges

- REST API endpoint inconsistencies between documentation and actual implementation — requiring close collaboration with developers for clarification

- Handling authentication token expiry during long test runs — implemented token refresh mechanism in test setup
- Managing test data setup and cleanup in MongoDB to ensure test independence — implemented `@BeforeMethod` and `@AfterMethod` hooks for data management
- Integrating Cucumber with existing TestNG+Selenium framework required careful configuration of runner classes and step definition organization

6.4 Learnings from Challenges

Each challenge was resolved through systematic debugging, analyzing error logs, consulting documentation, and seeking guidance from the SME. These experiences reinforced the importance of patience, methodical problem-solving, and maintaining clean, well-documented test code.

Chapter 7

Topics to Explore Further

7.1 Advanced Test Automation

- Playwright and Cypress as modern alternatives to Selenium for faster, more reliable UI testing
- Visual regression testing using tools like Applitools or Percy for UI consistency validation
- Contract testing using Pact for validating API contracts between microservices
- AI-powered test generation and self-healing locators for next-generation test automation

7.2 UI Automation and BDD

- JMeter and Gatling for performance testing and load testing of web applications
- OWASP ZAP and Burp Suite for automated security testing and vulnerability scanning
- Chaos engineering principles applied to test infrastructure resilience

7.3 CI/CD and DevOps for QA

- Jenkins and GitHub Actions for building comprehensive CI/CD pipelines with automated test gates
- Docker-based test environments for consistent, reproducible test execution across teams
- Infrastructure as Code (Terraform) for provisioning test environments on demand

7.4 Agile and Project Skills

- Testing serverless applications on AWS Lambda and API Gateway
- Advanced Terraform practices: modules, workspaces, remote state management, CI/CD integration
- Monitoring and observability tools (Grafana, CloudWatch) for production test validation

Chapter 8

Conclusion

The internship at EPAM Systems was a rigorous, hands-on training experience that covered the full breadth of modern software test automation through six structured learning modules and a team-based Agile project. Starting from Java fundamentals and progressing through design patterns, TestNG, REST Assured, Selenium, and BDD Cucumber, the program built competency incrementally before applying all skills in a real project environment.

The team project — Test Automation for Travel Agency Web Application — provided invaluable experience working in an industry-grade Agile team of 10 engineers. Creating test strategy and test plan documents, building API and UI automation frameworks, and participating in sprint ceremonies gave a realistic preview of professional QA engineering work.

The internship developed demonstrable, hands-on competency in Java-based test automation, REST Assured for API testing, Selenium WebDriver for UI automation, TestNG for test management, Cucumber for BDD, and Agile project delivery. These skills form a strong foundation for a career in software quality assurance and test automation engineering.

This internship has clearly shaped a career direction toward software test automation and quality engineering. The structured, module-driven approach adopted by EPAM Systems — building skills progressively before applying them in team projects — proved to be an exceptionally effective model for professional development.

Sincere gratitude is expressed to EPAM Systems for providing a training program that genuinely challenges participants to build real skills, to Mr. Praveen Eruvanti for his expert mentorship, to Mr. Harishankar Chandrasekharan for his Scrum leadership, and to Mr. Masood Ahmed for his guidance from the university side.

Sincere gratitude is expressed to EPAM Systems for providing a training program that genuinely challenges participants to build real skills with real tools, and to the guide Prerna Sharma for consistent mentorship, clear assignment guidance, thorough review feedback, and professional encouragement throughout the internship.

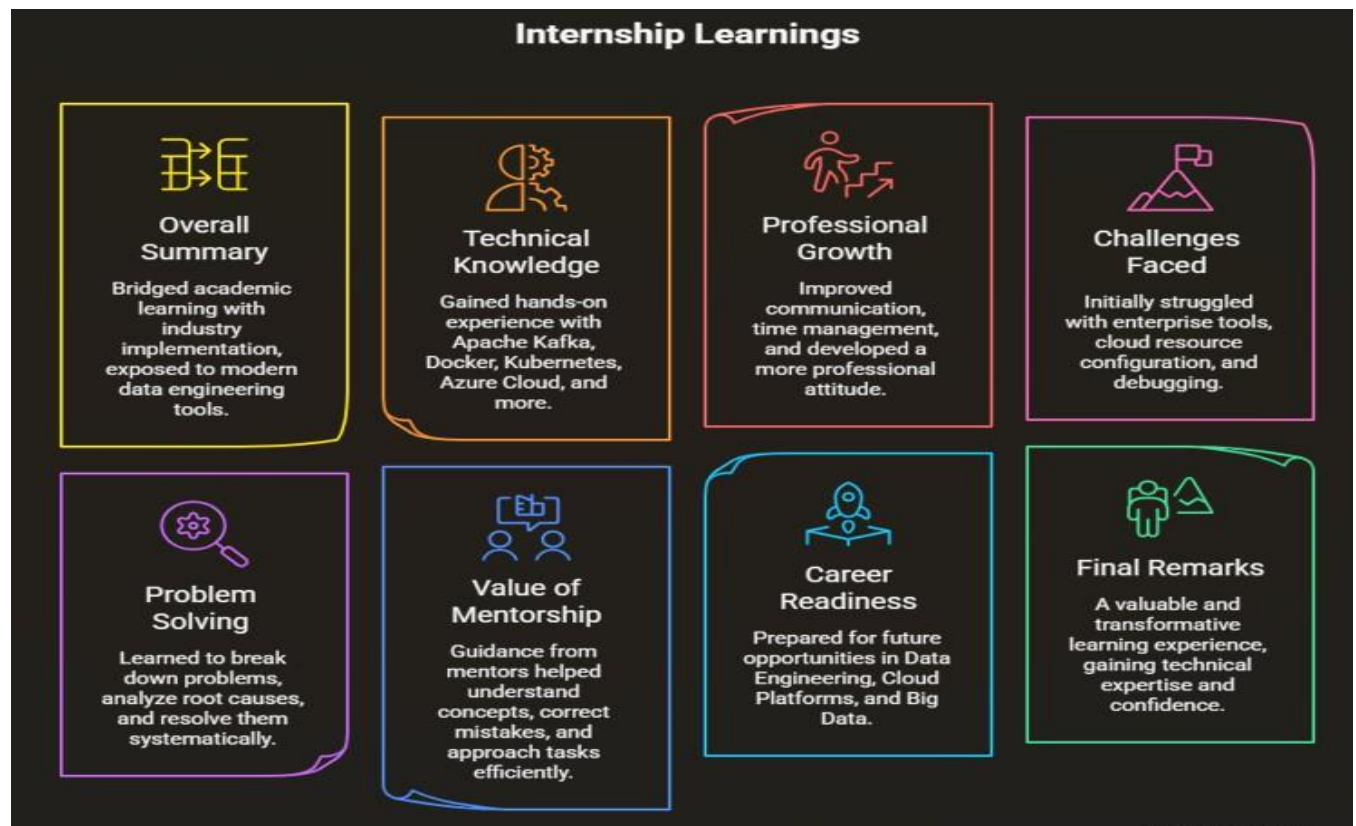


Figure 8.1: Internship Learnings Summary

Appendices

Appendix A – Module Submission Details

All six learning module capstone projects were submitted via the EPAM Learn portal. The team project code was maintained in a shared Git repository with proper branching strategy and pull request reviews.

Appendix B – Project Infrastructure Used

The Travel Agency project used the following infrastructure components for deployment and test execution:

- Kubernetes cluster (managed via KubeRocketCI) for backend deployment
- AWS S3 bucket for frontend static website hosting
- MongoDB database for application data persistence
- ArgoCD for GitOps-based continuous deployment
- Docker containers for application packaging and deployment
- Maven-based test automation framework for CI/CD integration

Appendix C – Test Best Practices Followed

The internship enforced strict quality assurance best practices throughout. The key practices followed were:

- Test cases were traceable to requirements — every user story had corresponding automated tests
- Test data was managed independently — setup and teardown ensured no test interference
- Page Object Model was enforced — no direct element interactions in test methods
- Code reviews were mandatory — all test code went through pull request review before merge
- Test reports were generated after every run — providing clear visibility into test health

Appendix D – Abbreviations

- API – Application Programming Interface
- MongoDB database for application data persistence
- CI/CD – Continuous Integration / Continuous Deployment
- Docker containers for application packaging and deployment
- REST – Representational State Transfer
- QA – Quality Assurance
- UI – User Interface
- MVP – Minimum Viable Product
- PoC – Proof of Concept
- AWS – Amazon Web Services
- S3 – Simple Storage Service
- SUT – System Under Test

References

1. Selenium WebDriver Documentation. Available at: <https://www.selenium.dev/documentation/>
2. REST Assured Documentation. Available at: <https://rest-assured.io/>
3. TestNG Documentation. Available at: <https://testng.org/doc/documentation-main.html>
4. Cucumber Documentation. Available at: <https://cucumber.io/docs/>
5. Spring Boot Documentation. Available at: <https://spring.io/projects/spring-boot>
6. Maven Documentation. Available at: <https://maven.apache.org/guides/>
7. React Documentation. Available at: <https://react.dev/>
8. MongoDB Documentation. Available at: <https://www.mongodb.com/docs/>
9. AWS Documentation. Available at: <https://docs.aws.amazon.com/>
10. Kubernetes Documentation. Available at: <https://kubernetes.io/docs/>
11. Docker Documentation. Available at: <https://docs.docker.com/>
12. ArgoCD Documentation. Available at: <https://argo-cd.readthedocs.io/>
13. KubeRocketCI Documentation. Available at: <https://docs.kuberocketci.io/>
14. EPAM Systems Internal Training Modules and QA Workshop Materials.
15. Mentor Guidance and Code Review Feedback provided by Praveen Eruvanti, EPAM Systems.

